Projet C# console — Gestion de factures pour une petite entreprise

Contexte

L'entreprise StartDev est une petite entreprise qui ne possède pas encore de base de données.

Les commerciaux enregistrent actuellement les nouveaux clients dans des fichiers Excel. Ils transmettent ensuite ces données à l'équipe de développement au format CSV.

StartDev souhaite disposer d'une application console en C# permettant :

- d'importer les nouveaux clients depuis un fichier CSV ;
- d'importer les entreprises depuis un fichier CSV ;
- de sauvegarder automatiquement ces données au format JSON ;
- d'afficher les clients et les entreprises ;
- de créer une facture ;
- de calculer automatiquement les montants HT, TVA et TTC ;
- de générer un fichier texte contenant la facture ;
- d'afficher un carnet de contacts regroupant toutes les personnes enregistrées dans l'application.

Objectif

Vous devez développer une application console en C# nommée :

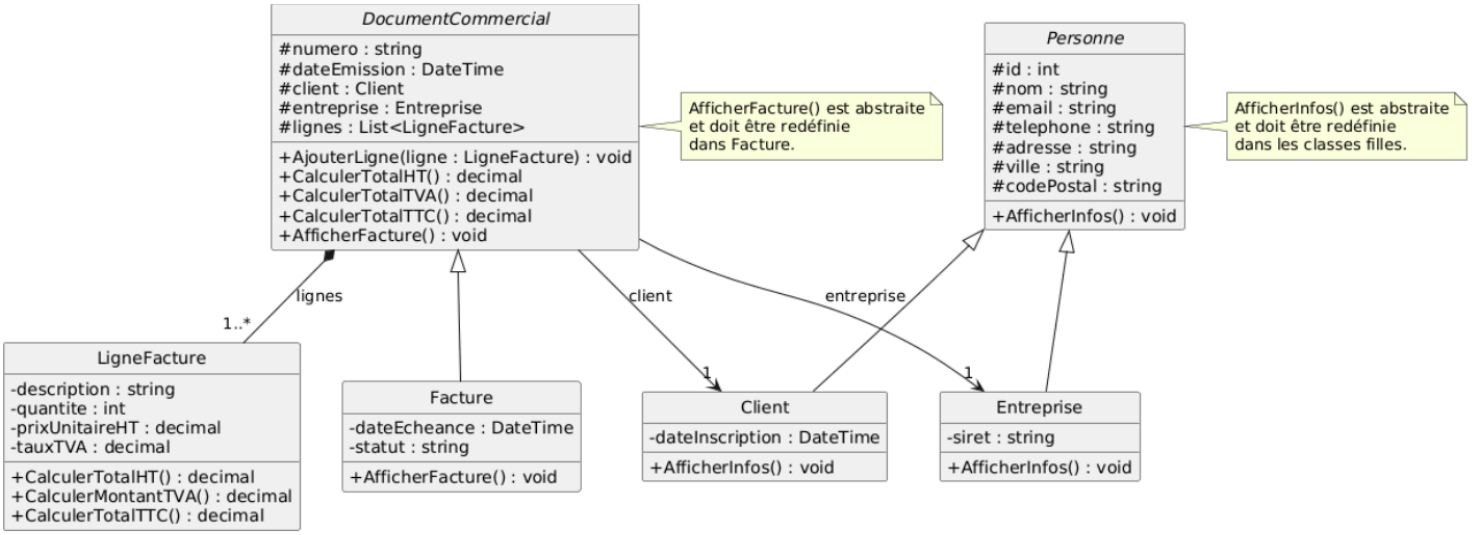
`ProjetFacturationConsole`

Deux diagrammes UML vous sont fournis :

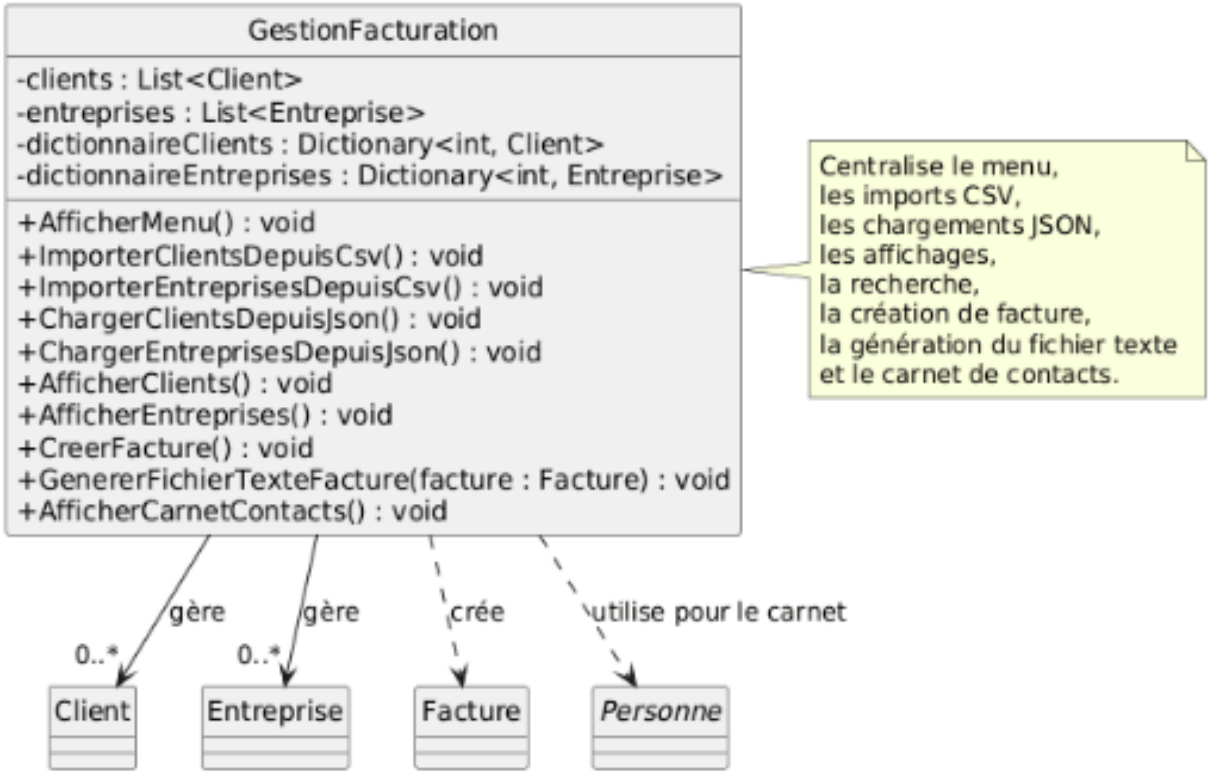
- un diagramme UML du modèle métier ;
- un diagramme UML de la classe de gestion de l'application.

Le projet doit être réalisé en console.

UML du modèle métier



UML classe de gestion de l'application



1. Travail demandé

L'application doit permettre :

- d'importer les clients depuis `clients.csv` ;
- d'importer les entreprises depuis `entreprises.csv` ;
- de générer automatiquement `clients.json` et `entreprises.json` après l'import ;
- d'afficher les clients ;
- d'afficher les entreprises ;
- de créer une facture ;
- de calculer les montants HT, TVA et TTC ;
- de générer un fichier texte contenant la facture ;
- d'afficher un carnet de contacts regroupant clients et entreprises ;
- d'utiliser les notions de POO vues en cours ;
- de contenir quelques tests unitaires simples.

2. Organisation générale du projet

Le projet doit être organisé de manière à limiter le code dans `Program.cs`.

Dans `Main()`, vous devez :

- créer un objet de la classe `GestionFacturation` ;
- appeler la méthode `AfficherMenu()`.

Le reste du traitement doit être réalisé dans des méthodes.

Les méthodes indiquées dans le sujet constituent le minimum attendu.

Vous êtes autorisés à ajouter d'autres méthodes si cela vous permet :

- d'éviter la répétition de code ;
- de rendre votre programme plus clair ;
- de mieux organiser vos traitements.

Toute méthode supplémentaire doit avoir un nom clair et un rôle précis.

3. Étapes de réalisation

Étape 1 — Récupérer le dépôt et préparer le projet

Vous devez :

- Accepter le devoir sur GitHub Classroom. Le lien GitHub Classroom est le suivant :

  https://classroom.github.com/a/e6pPLtXY

  - Se connecter à son compte GitHub.
  - Cliquer sur le bouton vert “Accept this assignment”.
  - Accepter ensuite l’invitation reçue sur l’adresse mail liée au compte GitHub.
  - Cloner enfin le dépôt.
- À la racine du dépôt GitHub Classroom, créer un fichier `.gitignore` contenant au minimum :

  ```
  bin/
  obj/
  .vs/
  ```

- Créer ensuite, dans ce dépôt, un dossier nommé `ProjetFacturationConsole`, puis ouvrir ce dossier dans VS Code.
- Dans le terminal de VS Code, créer la solution :

  ```
  dotnet new sln
  ```

- Créer ensuite le projet de l’application :

  ```
  dotnet new console -o ProjetFacturationConsole.App --use-program-main
  dotnet sln add .\ProjetFacturationConsole.App\ProjetFacturationConsole.App.csproj
  ```

- Créer le projet de tests :

  ```
  dotnet new mstest -o ProjetFacturationConsole.Tests

  dotnet sln add
  .\ProjetFacturationConsole.Tests\ProjetFacturationConsole.Tests.csproj

  dotnet add .\ProjetFacturationConsole.Tests\ProjetFacturationConsole.Tests.csproj
  reference .\ProjetFacturationConsole.App\ProjetFacturationConsole.App.csproj
  ```

- Faire ensuite un premier commit, puis créer une branche de travail :

  ```
  git add .
  git commit -m "Initialisation de la solution"
  git switch -c projet-facturation
  ```

À la fin de cette étape, vous devez obtenir une structure proche de celle-ci :

```
repo-github-classroom/
├── .gitignore
└── ProjetFacturationConsole/
    ├── ProjetFacturationConsole.sln
    ├── ProjetFacturationConsole.App/
    │   ├── ProjetFacturationConsole.App.csproj
    │   ├── Program.cs
    │   ├── Client.cs
    │   ├── Entreprise.cs
    │   ├── Facture.cs
    │   └── ...
    └── ProjetFacturationConsole.Tests/
        ├── ProjetFacturationConsole.Tests.csproj
        ├── UnitTest1.cs
        └── ...
```

Conseil : vérifiez dès le début que le dépôt est bien cloné et que votre projet est bien créé au bon endroit.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 2 — Créer la structure des classes dans le projet d'application

Vous devez créer uniquement :

- les classes ;
- les attributs avec le niveau d'accès indiqué dans le diagramme ;
- les propriétés publiques ;
- les constructeurs qui vous semblent nécessaires.

À cette étape, vous n'avez pas besoin de coder les autres méthodes.

Conseil : dès que vous créez une classe, faites un test manuel simple dans Main() pour vérifier qu'elle est reconnue par le projet et qu'il n'y a pas d'erreur de compilation.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 3 — Programmer les méthodes de base des classes métier

Méthodes concernées à cette étape :

- `Client.AfficherInfos()`
- `Entreprise.AfficherInfos()`
- `LigneFacture.CalculerTotalHT()`

- `LigneFacture.CalculerMontantTVA()`
- `LigneFacture.CalculerTotalTTC()`
- `DocumentCommercial.AjouterLigne(LigneFacture ligne)`
- `DocumentCommercial.CalculerTotalHT()`
- `DocumentCommercial.CalculerTotalTVA()`
- `DocumentCommercial.CalculerTotalTTC()`
- `Facture.AfficherFacture()`

Pour le détail du comportement attendu, voir la partie Règles de gestion et méthodes à réaliser.

Conseil : dès qu'une méthode est créée, faites un test manuel simple en créant un objet et en appelant la méthode avant de continuer.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 4 — Importer les données depuis les CSV et générer les JSON

Méthodes concernées à cette étape :

- `GestionFacturation.ImporterClientsDepuisCsv()`
- `GestionFacturation.ImporterEntreprisesDepuisCsv()`

Pour le détail du comportement attendu, voir la partie Règles de gestion et méthodes à réaliser.

Dans cette étape, vous devez :

- lire les fichiers CSV ;
- créer les objets correspondants ;
- remplir les listes ;
- remplir les dictionnaires (id client en clé et objet client en valeur, idem pour entreprise) ;
- générer automatiquement les fichiers JSON.

Conseil : après chaque import, faites un test manuel simple en affichant quelques données pour vérifier que l'import fonctionne correctement.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 5 — Charger et afficher les données

Méthodes concernées à cette étape :

- `GestionFacturation.ChargerClientsDepuisJson()`
- `GestionFacturation.ChargerEntreprisesDepuisJson()`

- `GestionFacturation.AfficherClients()`
- `GestionFacturation.AfficherEntreprises()`

Pour le détail du comportement attendu, voir la partie Règles de gestion et méthodes à réaliser.

Conseil : faites un test manuel simple de l'affichage dans Main() dès que ces méthodes sont codées.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 6 — Créer une facture et générer le fichier texte

Méthodes concernées à cette étape :

- `GestionFacturation.CreerFacture()`
- `GestionFacturation.GenererFichierTexteFacture(Facture facture)`

Pour le détail du comportement attendu, voir la partie Règles de gestion et méthodes à réaliser.

Cette étape réutilise aussi les méthodes déjà codées dans :

- `LigneFacture`
- `DocumentCommercial`
- `Facture`

Conseil : faites d'abord un test manuel simple avec une facture de test contenant seulement deux lignes avant de passer au menu complet.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 7 — Gérer les erreurs avec des exceptions

Méthodes concernées à cette étape :

- `GestionFacturation.ImporterClientsDepuisCsv()`
- `GestionFacturation.ImporterEntreprisesDepuisCsv()`
- `GestionFacturation.CreerFacture()`

Pour le détail du comportement attendu, voir la partie Règles de gestion et méthodes à réaliser.

Vous devez gérer au minimum les cas suivants :

- quantité invalide ;
- prix invalide ;

- date invalide ;
- date d'inscription supérieure à la date du jour.

Conseil : faites quelques tests manuels simples avec des données volontairement incorrectes pour vérifier que le programme ne se ferme pas brutalement.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 8 — Afficher le carnet de contacts

Méthode concernée à cette étape :

- `GestionFacturation.AfficherCarnetContacts()`

Pour le détail du comportement attendu, voir la partie Règles de gestion et méthodes à réaliser.

Conseil : faites un test manuel simple avec quelques clients et quelques entreprises pour vérifier que l'affichage dépend bien du type réel de l'objet.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 9 — Créer le menu général de l'application

Méthode concernée à cette étape :

- `GestionFacturation.AfficherMenu()`

Pour le détail du comportement attendu, voir la partie Règles de gestion et méthodes à réaliser.

Conseil : testez le menu progressivement, option par option, sans attendre d'avoir tout terminé.

Rappel : à la fin de cette étape, faites un commit avec un message significatif, puis faites un push sur GitHub.

Étape 10 — Ajouter les tests unitaires

Méthodes concernées à cette étape :

- tests de `LigneFacture.CalculerTotalHT()`
- tests de `LigneFacture.CalculerTotalTTC()`
- tests de `DocumentCommercial.CalculerTotalTTC()` via une `Facture`

Vous devez écrire les tests demandés dans le sujet à l'intérieur du projet de test créé préalablement.

4. Règles de gestion et méthodes à réaliser

4.1 Classe `Personne`

`Personne` est une classe abstraite.

Règle de gestion

`AfficherInfos()` est abstraite et doit être redéfinie dans les classes filles.

4.2 Classe `Client`

Règle de gestion

`AfficherInfos()` doit afficher exactement les informations suivantes dans cet ordre :

`id - nom - email - telephone - adresse - ville - codePostal - dateInscription`

Exemple d'affichage attendu

1 - Paul Durand - paul.durand@mail.fr - 0611223344 - 12 rue Victor Hugo - Amiens - 80000 - 12/03/2024

Validation à respecter

- la date d'inscription d'un client doit être inférieure ou égale à la date du jour.

4.3 Classe `Entreprise`

Règle de gestion

`AfficherInfos()` doit afficher exactement les informations suivantes dans cet ordre :

`id - nom - email - telephone - adresse - ville - codePostal - siret`

Exemple d'affichage attendu

1 - TechNova - contact@technova.fr - 0322000001 - 25 rue des Lilas - Amiens - 80000 - 12345678900011

4.4 Classe `LigneFacture`

Règles de gestion

`CalculerTotalHT()` retourne : `quantite × prixUnitaireHT`

`CalculerMontantTVA()` retourne : `CalculerTotalHT() × tauxTVA / 100`

`CalculerTotalTTC()` retourne : `CalculerTotalHT() + CalculerMontantTVA()`

4.5 Classe `DocumentCommercial`

`DocumentCommercial` est une classe abstraite.

Règles de gestion

`AjouterLigne(LigneFacture ligne)` ajoute la ligne reçue dans la liste des lignes.

`CalculerTotalHT()` retourne la somme des totaux HT de toutes les lignes.

`CalculerTotalTVA()` retourne la somme des montants de TVA de toutes les lignes.

`CalculerTotalTTC()` retourne la somme des totaux TTC de toutes les lignes.

`AfficherFacture()` est abstraite et doit être redéfinie dans `Facture`.

Pour éviter de recopier deux fois le même code, vous pouvez créer une méthode supplémentaire qui construit le texte complet de la facture.

Ce texte pourra ensuite être :

- affiché dans la console dans la méthode `AfficherFacture()` ;
- écrit dans un fichier dans la méthode `GenererFichierTexteFacture(...)`.

Pour construire progressivement un texte en C#, vous pouvez utiliser `StringBuilder` qui permet de construire des chaînes de caractères sur plusieurs lignes.

Exemple :

```
using System.Text;

StringBuilder sb = new StringBuilder();

sb.AppendLine("Ligne 1");

sb.AppendLine("Ligne 2");

sb.AppendLine("Ligne 3");

string texte = sb.ToString();
```

4.6 Classe `Facture`

Règles de gestion

Une facture doit contenir :

- le numéro de facture ;
- la date d'émission ;
- la date d'échéance ;
- le statut ;
- les informations de l'entreprise ;
- les informations du client ;
- la liste des lignes de facture ;
- le total HT ;
- le total TVA ;
- le total TTC.

La date d'échéance d'une facture doit être égale à la date d'émission plus 30 jours.

Une facture doit contenir au minimum deux lignes.

Affichage attendu pour `AfficherFacture()`

```
FACTURE

Numéro : F2026-001

Date d'émission : 15/05/2026

Date d'échéance : 14/06/2026

Statut : Brouillon

Entreprise :
1 - TechNova - contact@technova.fr - 0322000001 - 25 rue des Lilas - Amiens - 80000 - 12345678900011

Client :
2 - Paul Durand - paul.durand@mail.fr - 0611223344 - 12 rue Victor Hugo - Amiens - 80000 - 12/03/2024

Lignes :
1. Développement module connexion - Qté : 2 - PU HT : 150 - TVA : 20 - Total HT : 300 - Total TTC : 360
2. Maintenance corrective - Qté : 1 - PU HT : 80 - TVA : 10 - Total HT : 80 - Total TTC : 88

Total HT : 380

Total TVA : 68

Total TTC : 448
```

4.7 Classe `GestionFacturation`

Cette classe centralise le fonctionnement de l'application.

Règles de gestion des méthodes

`AfficherMenu()`

Cette méthode doit afficher exactement le menu suivant :

1 - Importer les clients depuis le CSV
2 - Importer les entreprises depuis le CSV
3 - Afficher les clients
4 - Afficher les entreprises
5 - Créer une facture
6 - Afficher le carnet de contacts
0 - Quitter

Puis elle doit permettre à l'utilisateur de choisir une option et d'exécuter l'action correspondante.

`ImporterClientsDepuisCsv()`

Cette méthode doit :

- lire `clients.csv` ;
- créer les objets `Client` ;
- remplir la liste `clients` ;
- remplir le dictionnaire `dictionnaireClients` (clé = id; valeur = client);
- générer automatiquement `clients.json`.

`ImporterEntreprisesDepuisCsv()`

Cette méthode doit :

- lire `entreprises.csv` ;
- créer les objets `Entreprise` ;
- remplir la liste `entreprises` ;
- remplir le dictionnaire `dictionnaireEntreprises` (clé = id; valeur = entreprise) ;
- générer automatiquement `entreprises.json`.

`ChargerClientsDepuisJson()`

Cette méthode doit :

- lire `clients.json` ;
- recréer la liste `clients` ;
- recréer aussi `dictionnaireClients` à partir de la liste chargée.

`ChargerEntreprisesDepuisJson()`

Cette méthode doit :

- lire `entreprises.json` ;
- recréer la liste `entreprises` ;
- recréer aussi `dictionnaireEntreprises` à partir de la liste chargée.

`AfficherClients()`

Cette méthode doit :

- vérifier si la liste `clients` est vide ;
- si la liste est vide, appeler `ChargerClientsDepuisJson()`;
- afficher les clients à partir de la liste `clients`.

Format attendu :

id - nom

Exemple :

1 - Paul Durand
2 - Léa Martin
3 - Omar Petit

`AfficherEntreprises()`

Cette méthode doit :

- vérifier si la liste `entreprises` est vide ;
- si la liste est vide, appeler `ChargerEntreprisesDepuisJson()` ;
- afficher les entreprises à partir de la liste `entreprises`.

Format attendu :

id - nom

Exemple :
1 - TechNova
2 - DevSolutions

`CreerFacture()`

Cette méthode doit :

- vérifier si les clients sont chargés ; sinon appeler `ChargerClientsDepuisJson()`;
- vérifier si les entreprises sont chargées ; sinon appeler `ChargerEntreprisesDepuisJson()` ;
- afficher la liste des entreprises avec leur identifiant et leur nom ;
- demander l'identifiant d'une entreprise ;
- récupérer l'entreprise dans `dictionnaireEntreprises` ;
- afficher la liste des clients avec leur identifiant et leur nom ;
- demander l'identifiant d'un client ;
- récupérer le client dans `dictionnaireClients` ;
- demander la date d'émission ;
- calculer automatiquement la date d'échéance à 30 jours après la date d'émission ;
- demander les lignes de facture ;
- créer la facture ;
- afficher la facture ;
- demander confirmation ;
- appeler la méthode de génération du fichier texte.

Pour chaque ligne, l'utilisateur saisit :

- la description ;
- la quantité ;
- le prix unitaire HT ;
- le taux de TVA.

Après chaque ligne, le programme doit demander :

Voulez-vous ajouter une autre ligne ? (oui/non)

Avant de générer le fichier texte, le programme doit demander :

Confirmer la génération du fichier texte ? (oui/non)

`GenererFichierTexteFacture(Facture facture)`

Cette méthode doit générer un fichier texte contenant exactement le même texte que la méthode AfficherFacture() :

Le nom du fichier doit respecter le format suivant :

facture_NUMERO.txt

Exemple :

facture_F2026-001.txt

Pour générer le fichier texte, vous pouvez reprendre la même logique générale que pour l'écriture d'un fichier CSV :

- construire les lignes de texte avec un StringBuilder (voir les infos dans les règles de gestion de la méthode AfficherFacture);
- écrire ces lignes dans un fichier.

La différence est qu'ici le contenu est du texte libre destiné à être lu par un humain.
Vous n'avez donc pas besoin d'utiliser le séparateur `;`.

`AfficherCarnetContacts()`

Cette méthode doit :

- vérifier si les clients sont chargés ; sinon appeler `ChargerClientsDepuisJson()`;
- vérifier si les entreprises sont chargées ; sinon appeler `ChargerEntreprisesDepuisJson()` ;
- créer une collection de type `List<Personne>` ;
- y ajouter tous les clients ;
- y ajouter toutes les entreprises ;
- appeler `AfficherInfos()` sur chaque élément de la collection.

Cette méthode permet d'afficher un carnet de contacts unique pour l'entreprise.

Exemple d'affichage attendu

1 - Paul Durand - paul.durand@mail.fr - 0611223344 - 12 rue Victor Hugo - Amiens - 80000 - 12/03/2024
2 - Léa Martin - lea.martin@mail.fr - 0622334455 - 5 avenue Jean Jaurès - Beauvais - 60000 - 02/11/2023
1 - TechNova - contact@technova.fr - 0322000001 - 25 rue des Lilas - Amiens - 80000 - 12345678900011
2 - DevSolutions - contact@devsolutions.fr - 0322000002 - 8 boulevard Faidherbe - Lille - 59000 - 23456789000022

5. Fichiers fournis

Les fichiers suivants sont fournis dans le projet :

- `clients.csv`
- `entreprises.csv`

Le séparateur utilisé dans les fichiers CSV est le point-virgule.

L'ordre des colonnes dans les fichiers CSV respecte exactement l'ordre présenté ci-dessous.

Structure de `clients.csv`

Colonne	Contenu
1	id
2	nom
3	email
4	telephone
5	adresse
6	ville
7	codePostal
8	dateInscription

Structure de `entreprises.csv`

Colonne	Contenu
1	id

2	nom
3	email
4	telephone
5	adresse
6	ville
7	codePostal
8	siret

6. Contraintes techniques

Exceptions

Les erreurs doivent être gérées à l'aide d'exceptions.

Vous devez lever des exceptions lorsqu'une règle n'est pas respectée ou lorsqu'une donnée est invalide, puis intercepter ces exceptions pour afficher un message clair à l'utilisateur sans arrêter brutalement le programme.

Les cas suivants doivent être gérés au minimum :

- quantité invalide ;
- prix invalide ;
- date invalide ;
- date d'inscription supérieure à la date du jour.

Le programme ne doit pas se fermer brutalement dans ces cas.

Tests unitaires

Vous devez écrire les tests suivants :

- test du calcul du total HT d'une ligne ;
- test du calcul du total TTC d'une ligne ;
- test du total TTC d'une facture contenant plusieurs lignes.